

Vehicle Rental/ Booking application Flow:

1) Create a Spring Container

```
SpringApplication.run(VehicleRentalApplication.class,  
args);
```

1.1) @ComponentScan creates instances of

- VehicleRentalApplication
- DateUtil
- BookingRepo
- CustomerRepo
- VehicleRepo
- RentalService
- BookingClient
- RentalClient
- VehicleClient

1.2) @EnableAutoConfiguration

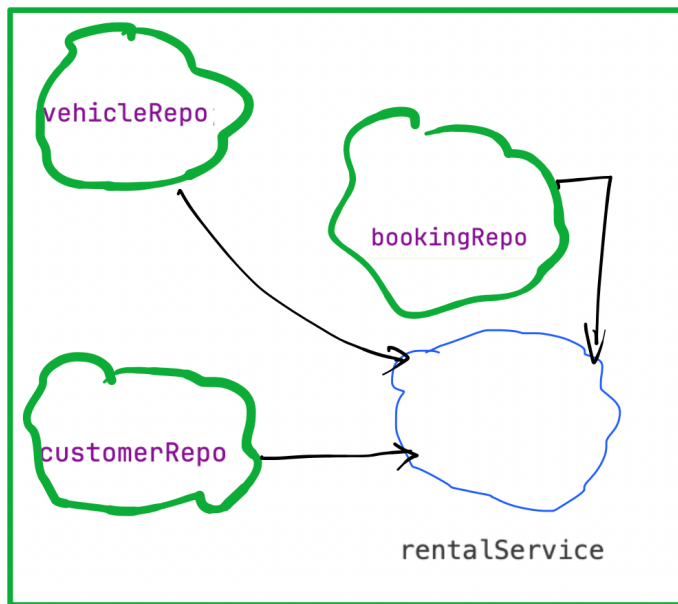
- Connection Pool using details present in application.properties
- EntityManagerFactory using ConnectionPool and entries in application.properties
- Injects EntityManager to all repo classes
- Injects repo to RentalService
- Injects DateUtil to RentalService
- Injects RentalService to all Client classes [CommandLineRunner]

2) Container is initialized

3) Invokes run() methods of CommandLineRunner classes depends on @Order

Invoking part:

Client -> Service → Repository classes → JPA -> JDBC → RDBMS



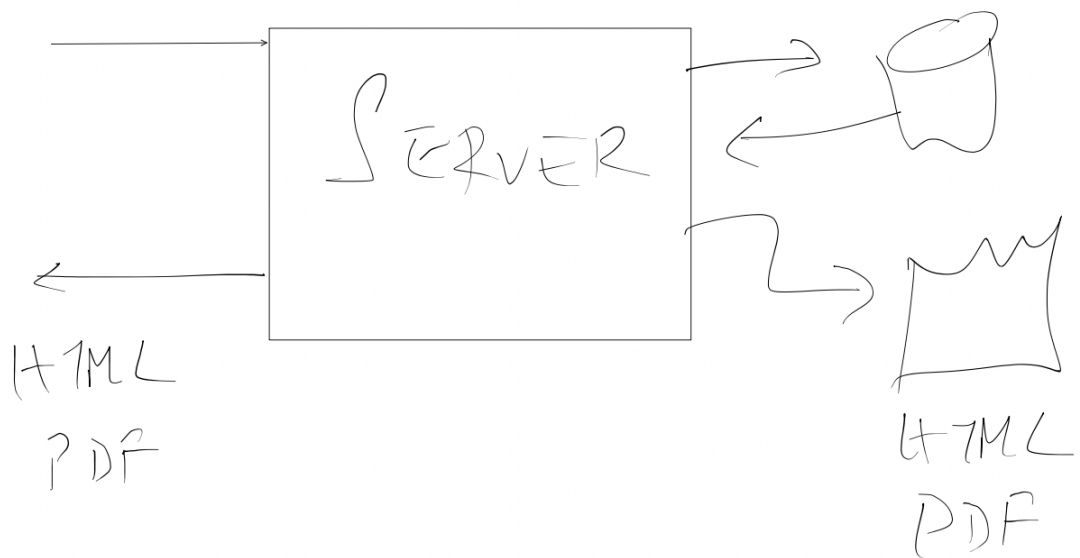
```

@Service
@RequiredArgsConstructor
public class RentalService {
    private final VehicleRepo vehicleRepo;
    private final CustomerRepo customerRepo;
    private final BookingRepo bookingRepo;
}
  
```

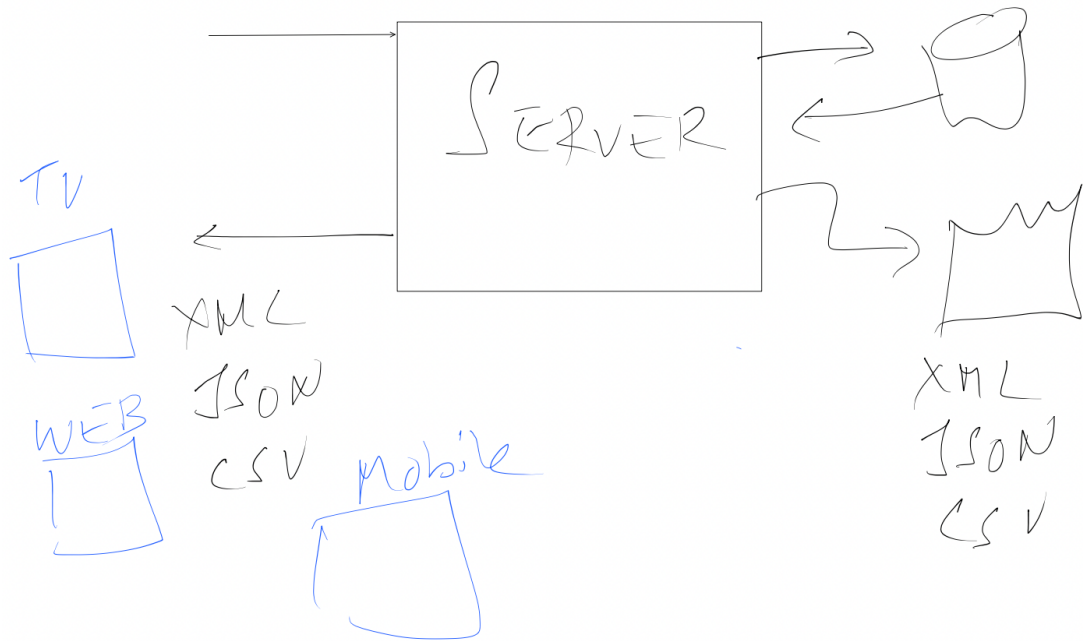
```

public class RentalService {
    public RentalService(VehicleRepo vehicleRepo, CustomerRepo customerRepo, BookingRepo bookingRepo) {
        this.vehicleRepo = vehicleRepo;
        this.customerRepo = customerRepo;
        this.bookingRepo = bookingRepo;
    }
}
  
```

SSA



CS4



REST API

<https://api.restaurant.com>

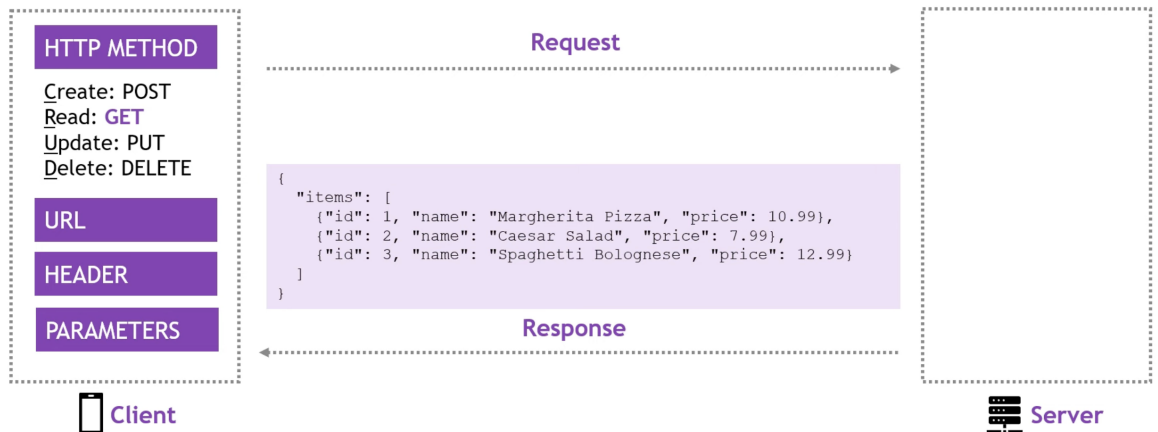
See what's available to order



REST API

<https://api.restaurant.com>

See what's available to order



REST API

<https://api.restaurant.com>

Place an order for Margherita Pizza (id #1)



REST API

<https://api.restaurant.com>

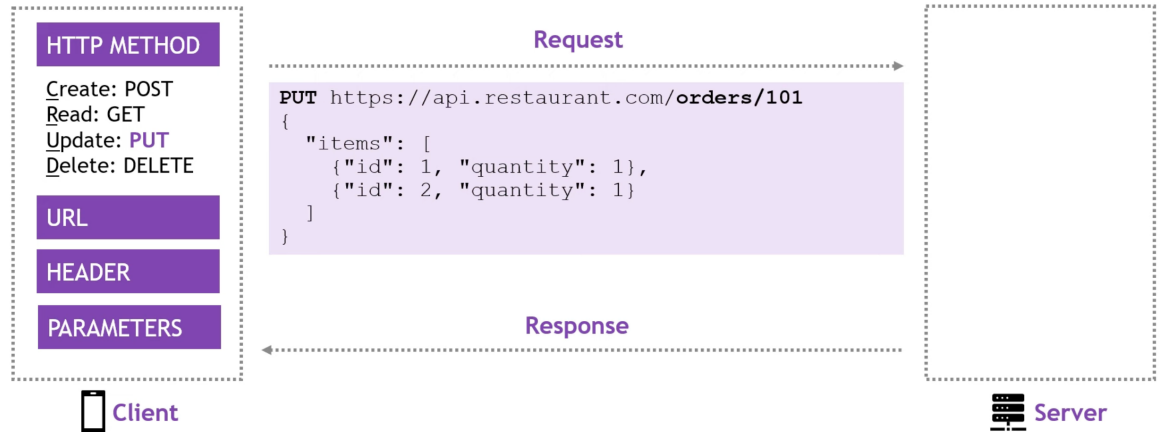
Place an order for Margherita Pizza (id #1)



REST API

<https://api.restaurant.com>

Update your order to add a Caesar Salad (id #2)



REST API

<https://api.restaurant.com>

Cancel your order

